

DSA4212 Project Report

Training a Language Model Within a Fixed Compute Budget: Which Model Size Performs Best on MAESTRO?

MAESTRO v3.0.0 | Random initialization | NVIDIA A10

Name: Teo Chloe

Student ID: A0286502E

Email: e1249216@u.nus.edu

Course: DSA4212

Date: 8 November 2026

Project repository

https://github.com/chloeheh/DSA4212-project-github

Download the MAESTRO v3.0.0 MIDI-only dataset from the official MAESTRO dataset page. The dataset is excluded from the GitHub repository because of its size.

1 Data and prediction task

This project used the MIDI-only MAESTRO v3.0.0 piano performances [1]. The supplied training, validation, and test splits contain 962, 137, and 177 pieces, with 33,293,600, 3,790,187, and 4,334,269 event tokens, respectively. According to the documentation, performances of the same composition are grouped in one split [2]. The split was preserved, but no independent audit of every possible overlap was carried out.

The tokenizer converts music into 376 instructions: 128 note starts, 128 note endings, 100 time shifts, 16 velocity bins, two pedal states, and BOS/EOS boundaries. Timing is rounded to 10 ms; longer pauses use multiple shifts of at most one second. At equal timestamps, note endings precede pedal changes and note starts. A velocity token precedes every note start. Only pedal state changes are emitted. This preserves expressive event order while discarding some timing/velocity detail and other MIDI metadata. A token is not a note, and 1,024 tokens cover different musical durations in busy and quiet passages.

The notebook checked token ranges, lengths, and piece boundaries, and decoded one source piece to inspect the tokenizer. The decoded piece was a tokenizer check, not a model-generated sample. Storage details and preparation steps are in the reproduction guide. The results measure event prediction, not audio reconstruction.

2 Research question, models, and evaluation criteria

Objective. The aim was to train models from random weights to predict the next music event and find the *best performance within a fixed training-time budget per run.* The project investigated model size rather than aiming to build a production music generator. The Smol Training Playbook recommends defining the purpose before choosing a model [3]. This practitioner guide helped frame the question; it was not treated as peer-reviewed evidence of model performance.

Initial hypothesis. Larger models were expected to achieve lower validation loss because they could learn more complex musical patterns. This hypothesis was tested under the same **two-hour GPU-time budget** for each size. It was an expectation to test, not a result the experiment had to produce. The initial comparison focused on model size. The baseline observations then motivated the follow-up question in Sections 6–7.

Table 1: Dense decoder-only GPT variants. Parameter counts include learned position embeddings; shared token and output weights are counted once.

Model	Layers	Heads	Width	Total parameters
Small	6	4	256	5,080,320
Medium	8	8	512	25,891,328
Large	12	12	768	86,029,056

Each run used one NVIDIA A10 GPU on Modal, with 24 GB of memory and BF16 support [4]. The models were trained sequentially. All used a vocabulary of 376 tokens, a context length of 1,024 tokens, and an effective batch size of 32 sequences. The labels small, medium, and large refer to this project's configurations, not pretrained GPT-2 variants.

Primary metric and checkpoint selection

For target event y_i and its earlier context c_i, the average negative log likelihood (cross-entropy loss) and perplexity are

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid c_i), \qquad \text{PPL} = \exp\{\mathcal{L}(\theta)\}. \tag{1}$$

Loss is calculated using natural logarithms and is measured in nats/token. Lower loss and perplexity indicate better predictions. Unlike accuracy, loss rewards a higher probability for the correct event, even when it is not the top prediction. It gives no special credit for a musically similar but incorrect pitch and does not measure listening quality.

Each validation check averaged 20 batches of four 1,024-token windows, giving 81,920 target positions, some of which could overlap. Evaluation seeds were reset so that all models were checked on the same windows. Dropout and gradient calculations were disabled during evaluation.

A checkpoint is a saved model state. The selected checkpoint, best.pt, had the lowest recorded validation loss within the budget; ckpt.pt stored the latest state. Test loss was not used for selection. Every run received the full budget, even if its best result occurred earlier. Final losses were also examined for signs of later overfitting. Section 9 describes the full-coverage evaluation.

3 Adapting nanoGPT to MAESTRO

nanoGPT was chosen for its compact PyTorch model and training code [5]. Its clear structure made data loading, optimization, and checkpointing easy to inspect and allowed model size to vary within the same framework. The choice was based on clarity and reuse; no claim is made that it was the fastest implementation.

The decoder learns token and absolute position embeddings. Causal attention uses earlier tokens while masking future targets [6]. Blocks use pre-layer normalization, residual connections, and GELU feed-forward networks with four times the hidden width. Token embeddings and the output head share weights. All models were initialized with random weights.

The loader memory-maps token files and samples uniformly over valid within-piece window starts. A piece with $L > 1{,}024$ tokens provides $L - 1{,}024$ starts, so longer pieces receive more sampling opportunities. Inputs are $t_0, \ldots, t_{1023}$ and targets are $t_1, \ldots, t_{1024}$. Neither crosses piece boundaries; shorter pieces cannot supply full training windows.

The loader validates offsets, lengths, and token IDs and overrides the inherited vocabulary default with 376. Architecture configurations and environment details are recorded in the repository.

Table 2: Shared training settings; the baseline uses dropout 0.1.

Optimizer	AdamW	Weight decay	0.1
(β_1, β_2)	$(0.9, 0.95)$	Gradient clipping	Norm 1.0
Peak / minimum LR	3×10^{-4} / 3×10^{-5}	Seed	1337
Precision	BF16 autocast	Effective batch	32 sequences
Context	1,024 tokens	Targets/update	32,768

AdamW adapts updates using past gradients and applies weight decay separately [7]. In these runs, matrix weights received decay; one-dimensional normalization parameters did not. Training used fused AdamW on CUDA and PyTorch scaled dot-product attention when available. Compilation was disabled, and compiler modes were not benchmarked.

The 7,200-second loop budget included validation and checkpoint writes. Setup, final evaluation, and saving added overhead, and any update in progress was allowed to finish at the time limit. The budget therefore covered approximately two hours of loop time, rather than an exact billed duration or a fixed number of FLOPs. Atomic checkpoints stored weights, optimizer and precision-scaler states, random states, token counts, and elapsed time. Resuming a run preserved its original budget. Modal Volumes retained outputs after client disconnection. The environment used Python 3.11, PyTorch 2.6.0, and NumPy 2.2.3.

4 Training faster within the time budget

Testing batch sizes while keeping 32 sequences per update

A microbatch is the set of sequences processed in one forward and backward pass. Microbatch sizes of 4, 8, 16, and 32 were benchmarked on the A10. Gradients were accumulated as needed to keep 32 sequences per optimizer update. Throughput measures the number of target tokens processed per second.

The benchmark timed complete updates, including data loading, transfer to the GPU, forward and backward passes, gradient clipping, and AdamW updates. After three warm-up steps, six measured steps were used to calculate the median update time.

The microbatch with the highest measured throughput below 85% GPU memory use was selected. Microbatch 32 performed best for all three sizes and was used for training, together with BF16 and fused optimization.

The benchmark compared only the tested batch sizes. It did not establish a global optimum or include periodic validation and checkpoint overhead.

Table 3: A10 batching measurements. Throughput is target tokens/s; memory is peak allocated GB, not total reserved/device usage.

Model	Batch 4	Batch 32	Speedup	Batch-32 GB
Small	424,395	603,065	1.42×	2.24
Medium	151,006	182,301	1.21×	5.85
Large	57,946	68,590	1.18×	13.22

What the same time budget means

All models followed the same learning-rate schedule based on elapsed time. Let $u = t/7{,}200$ be the fraction of the budget used. During the first $w = 0.02$ of the budget (about 144 seconds), the learning rate increased towards $\eta_{\max}$. After warm-up, the schedule was

$$\eta(u) = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2}\left[1 + \cos\left(\pi \frac{u - w}{1 - w}\right)\right], \quad u \geq w. \tag{2}$$

The code keeps the learning rate slightly above zero during warm-up. The models therefore had similar learning rates at the same elapsed time, despite completing different numbers of updates. Time spent on validation and checkpoint saving also counted towards the schedule.

Table 4: Actual baseline training exposure within approximately 7,200 seconds. Equivalent passes are tokens seen divided by 33,293,600; sampling is with replacement, not ordered epochs.

Model	Updates	Targets seen (billions)	Equivalent passes
Small	127,396	4.175	125.4
Medium	40,719	1.334	40.1
Large	15,430	0.506	15.2

Small processed about 8.3 times as many targets as large. The larger models therefore had fewer updates and less data exposure within the same time limit. Equal A10 time answers a GPU-rental-budget question, but does not isolate model size, equalize FLOPs, or measure efficiency at an equal token count. Sharing the learning rate and dropout improved experimental control without establishing the best settings for each size.

5 Baseline learning curves: dropout 0.1

Figure 1 shows 13 recorded evaluations for each model: before training, roughly every ten minutes, and at the end. Training loss was measured on fixed training windows with dropout turned off, rather than taken from a single training microbatch. The lower plots omit the initial evaluation and use the same scale to make later changes easier to compare. These baseline results guided the choice of the next experiment.

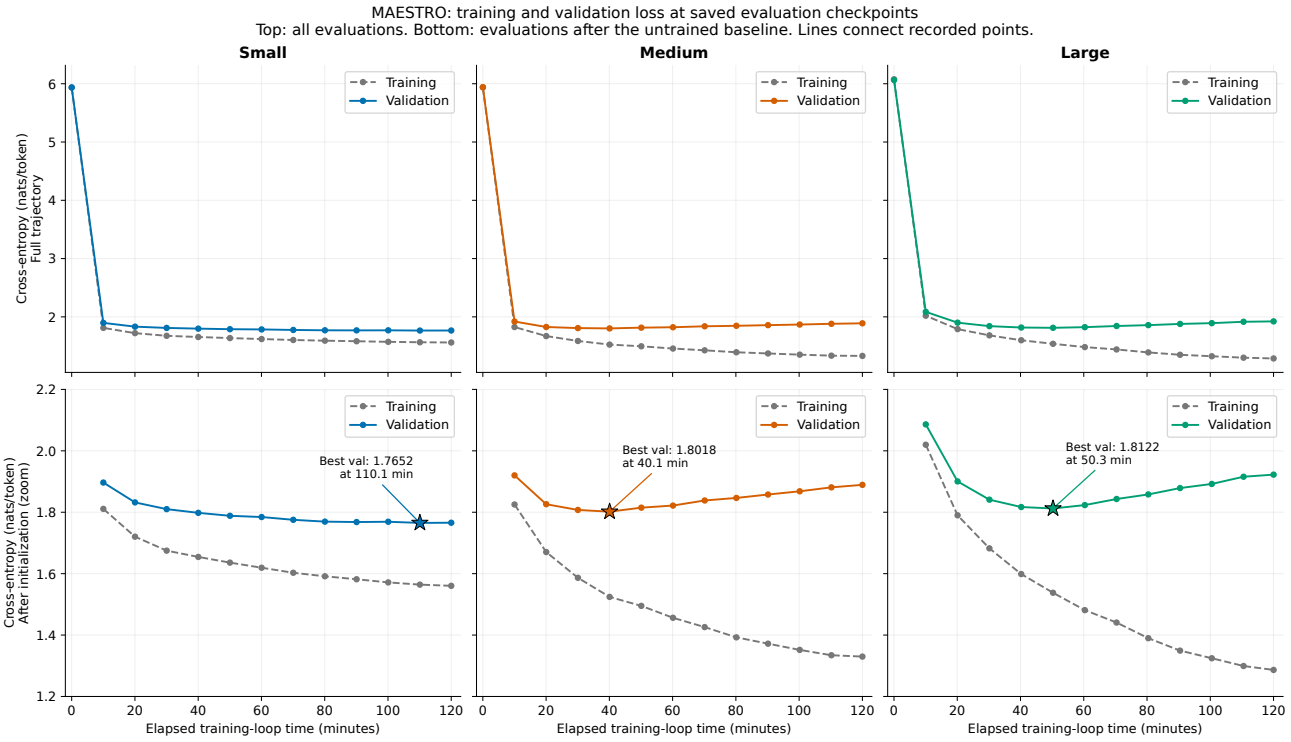


Figure 1: Baseline learning curves. Dashed grey: sampled training loss; solid colors: sampled validation loss. Stars mark best recorded validation checkpoints. The bottom row omits initialization. Lines connect recorded checks; they do not add observations.

Table 5: Baseline loss and best recorded time. All values are sampled cross-entropy in nats/token.

Model	Best val.	Best time (min)	Final train	Final val.
Small	**1.7652**	110.1	1.5603	1.7659
Medium	1.8018	40.1	1.3299	1.8892
Large	1.8122	50.3	1.2865	1.9224

All models improved quickly during the first ten minutes. Small then improved more slowly and levelled off near the end. Medium and large reached their best validation losses earlier. Their training losses then continued to fall while their validation losses rose. Large had the lowest final training loss but the highest final validation loss. This shows that fitting the training pieces well does not necessarily lead to better predictions on unseen pieces.

Based on the best recorded validation checkpoints, small ranked first, followed by medium and large. The differences were fairly small: medium and large had best-checkpoint perplexities about 3.7% and 4.8% higher than small. This does not mean that more parameters always hurt. It means that the larger models did not perform better with this dataset, implementation, schedule, and time limit.

6 What the baseline results led me to investigate

The initial hypothesis was not supported in these runs: small achieved the lowest recorded validation loss. More surprisingly, medium and large continued to improve on training data while becoming worse on validation data. Their final validation-minus-training gaps were 0.5593 and 0.6360, compared with 0.2055 for small. Goodfellow et al. describe overfitting as fitting training data well while failing to generalize to unseen data [8, Chapter 5]. Overfitting is therefore a likely explanation for the recorded curves. Repeated increases in validation loss, alongside falling, finite training losses, support this interpretation more than a numerical failure. They do not establish a single cause.

There was also a trade-off between model size and training time: medium and large had about 5.1 and 16.9 times as many parameters as small, but received fewer updates. Hoffmann et al. study the joint allocation of model size and training tokens under fixed FLOPs [9]. Their text-model setting, which uses fresh data, differs from this project's fixed corpus and GPU-time limit. Their token-to-parameter rule was therefore not applied, and these models are not described as "Chinchilla-optimal." Equal-token runs and learning-rate tuning would help separate the effects of capacity, exposure, and optimization.

Why might larger models perform worse on MAESTRO?

Capacity relative to musical diversity. The 33.3 million training tokens came from only 962 performances, and overlapping windows reused related passages. A larger model may have fitted piece-specific patterns that did not transfer to the composition-separated validation set. This is consistent with the trade-off between capacity and generalization [8, Chapter 5], but the loss curves do not prove memorization. Small saw more repeated targets yet generalized better, so repetition alone cannot explain the ranking.

Shared settings may suit sizes differently. The same dropout of 0.1 and weight decay may not have constrained every architecture equally. Stronger dropout later helped medium and large, suggesting that regularization contributed to the result without fully explaining it. The shared learning rate also decayed over far fewer updates for large. Research on allocating compute across model size and training tokens motivates separating optimization from capacity [9]. An equal-token comparison and tuning for each size would help test this explanation.

More parameters do not add musical context. All sizes received the same 1,024-token windows, which spanned varying musical durations. Larger models gained no additional history and had to learn transferable pitch and timing relations from the same representation. Music Transformer's piano results support investigating relative attention and augmentation [10], but do not establish a representation bottleneck here. These are possible contributors to the negative result, not experimentally isolated causes.

Follow-up question. These curves raised the question of whether stronger regularization could lower the best validation losses of medium and large. Choosing an earlier checkpoint already avoided their worse final weights, but could not improve the best result reached during training. The follow-up therefore aimed to improve prediction on unseen pieces, rather than simply reduce training loss. Validation results guided this investigation; test data were not used to choose the intervention.

7 Follow-up: can stronger dropout reduce overfitting?

Why regularization, and which kind? The rising validation losses motivated this follow-up; stronger dropout was not part of the initial model-size hypothesis. The baseline already used dropout 0.1 and AdamW weight decay 0.1, so the follow-up investigated whether stronger regularization would help.

Two relevant options are weight decay and dropout. Weight decay shrinks weights during optimization, discouraging large weight values. AdamW separates this shrinkage from the adaptive gradient update [7]. Dropout instead randomly removes activations during training, reducing reliance on particular feature combinations [11]. For drop probability p,

$$\widetilde{h} = \frac{m \odot h}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p). \tag{3}$$

Here, h is an activation and m is a random mask of zeros and ones. Remaining values are rescaled to keep their expected size unchanged. Dropout is disabled for evaluation. The shared probability affects embeddings, attention probabilities, attention outputs, and MLP outputs.

Dropout was increased from 0.1 to 0.2 while weight decay was kept unchanged. This allowed one setting to be tested within the remaining budget. The readings support considering both options, but do not establish which is best for this dataset. Weight norms were not measured, and stronger weight decay was not tested. The results therefore cannot establish that insufficient weight decay caused the problem or that dropout was the better remedy. The value 0.2 was a moderate trial, not a known optimum.

Controlled comparison. All three sizes were retrained from random initialization. Within each size, the architecture, seed 1337, AdamW settings, learning-rate schedule, context 1,024, batch 32, validation windows, evaluation frequency, and 7,200-second budget were kept unchanged. Only dropout and the output location changed. The launcher checked configurations against the saved baselines. Small was included to test whether an intervention motivated by medium and large also helped a model with less obvious overfitting. This added three two-hour runs, reusing the earlier batch-size benchmark.

Success criterion. Each size was compared with its own baseline using the lowest recorded validation loss. A smaller training–validation gap alone would not count as success if validation loss worsened. Final loss, the time of the best checkpoint, and tokens processed were secondary checks. The shared seed controlled initialization, but did not make dropout masks or training paths identical. This was a limited follow-up, not an exhaustive hyperparameter search.

8 Results after increasing dropout to 0.2

Figure 2 compares each size with its own baseline. The main criterion remains best recorded validation loss, rather than the final loss or the size of the training–validation gap.

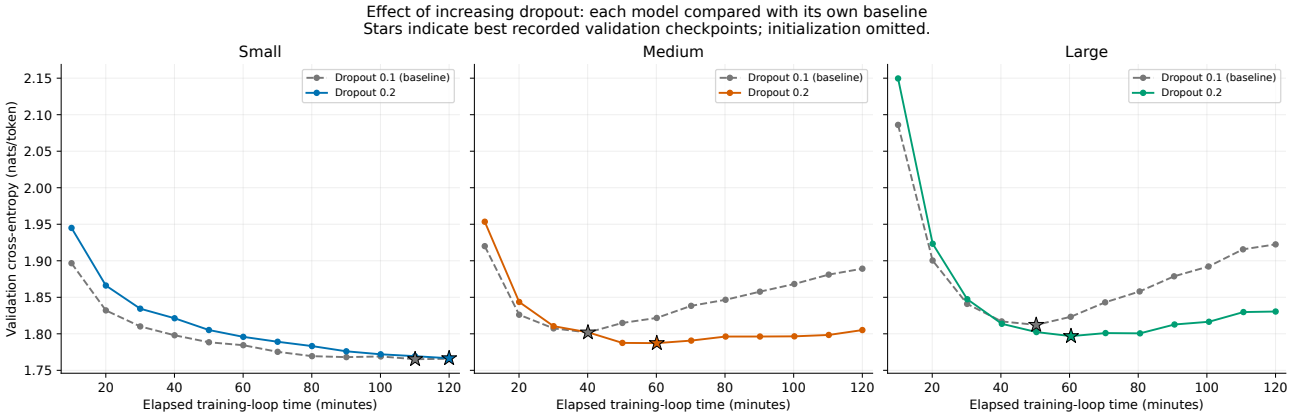


Figure 2: Follow-up prompted by the baseline curves: dropout 0.1 (grey dashed) versus 0.2 (colored solid). Each size is compared with its own baseline; stars mark its best recorded validation losses. Medium and large improve, while small does not.

Table 6: Sampled validation cross-entropy. Δ best is dropout 0.2 minus 0.1; negative means improvement.

	Best validation			Final validation	
Model	$p = 0.1$	$p = 0.2$	Δ best	$p = 0.1$	$p = 0.2$
Small	**1.7652**	1.7665	$+0.0013$	1.7659	1.7665
Medium	1.8018	**1.7872**	-0.0146	1.8892	1.8051
Large	1.8122	**1.7968**	-0.0154	1.9224	1.8305

Medium and large achieved best validation perplexities about 1.45% and 1.53% lower than their baselines. The best recorded times changed from 40.1 to 60.1 minutes for medium and from 50.3 to 60.4 minutes for large. Their final losses improved by 0.0840 and 0.0919 nats/token, respectively, much more than the gains at their best checkpoints. Stronger dropout therefore mainly limited late worsening, while also slightly improving the selected checkpoints. Validation loss still rose after each minimum, so the overfitting pattern remained.

For small, the final training–validation gap fell from 0.2055 to 0.1444, but its best validation loss worsened by 0.0013 (about 0.13% in perplexity). A smaller gap alone was therefore not a success. With one seed, this tiny change does not establish that stronger dropout reliably harms small.

In the follow-up, small, medium, and large processed 4.139B, 1.329B, and 0.507B targets, respectively. These were close to the baseline counts, so a major change in exposure is unlikely to explain the pattern. Small remained best. Whether different seeds, weight decay, learning rates, or equal-token budgets would change this ranking remains unresolved. Final testing addressed the narrower question of whether the validation-selected settings retained their ordering on unseen pieces. The six runs used about 12 training-loop GPU-hours, excluding benchmarking and overhead.

9 Final evaluation on unseen pieces

Selection and history. The selected settings were dropout 0.1 for small and dropout 0.2 for medium and large. Their checkpoints had the lowest recorded sampled-validation losses and were saved at about 110.1, 60.1, and 60.4 minutes, respectively. This compared selected configurations; the baseline with the same dropout for all sizes remained the cleaner size comparison. Validation-based selection assumes that the supplied split is informative about similar unseen piano pieces [8, Section 5.3].

The original test evaluation used dropout 0.2 for all sizes. After those results were viewed, the small entry was replaced with dropout 0.1, which had already been preferred by pre-test validation. Only evaluation of the small model was repeated. Its archived test loss was 1.6970, which also ranked first. The replacement did not change the ranking, but the revision means that the final comparison was not fully fixed before test access. These inspected test results must not guide future model choices.

Common scoring. Every token after the first token of each piece was scored once, including EOS and partial tails: 3,790,050 validation and 4,334,092 test targets. Dropout and gradient calculations were disabled, and padding was excluded. Consecutive blocks contained at most 1,024 targets, reset context, and never crossed pieces. Earlier targets in each block therefore had shorter histories. Validation was rescored using the same procedure as test. These full-coverage losses are not interchangeable with the sampled losses used for checkpoint selection.

Table 7: Selected checkpoints under common full-coverage scoring. Loss is cross-entropy in nats/token.

Model	Dropout	Full validation loss	Test loss	Test perplexity
Small	0.1	**1.7581**	**1.6935**	**5.4385**
Medium	0.2	1.7848	1.7179	5.5730
Large	0.2	1.7926	1.7277	5.6279

Medium and large had test perplexities about 2.5% and 3.5% higher than small's, respectively, despite having roughly five and seventeen times as many parameters. Small also achieved the lowest full-coverage validation loss, so changing the scoring coverage did not reverse the ranking. Lower test than validation losses may reflect differences in piece difficulty; testing does not improve model weights.

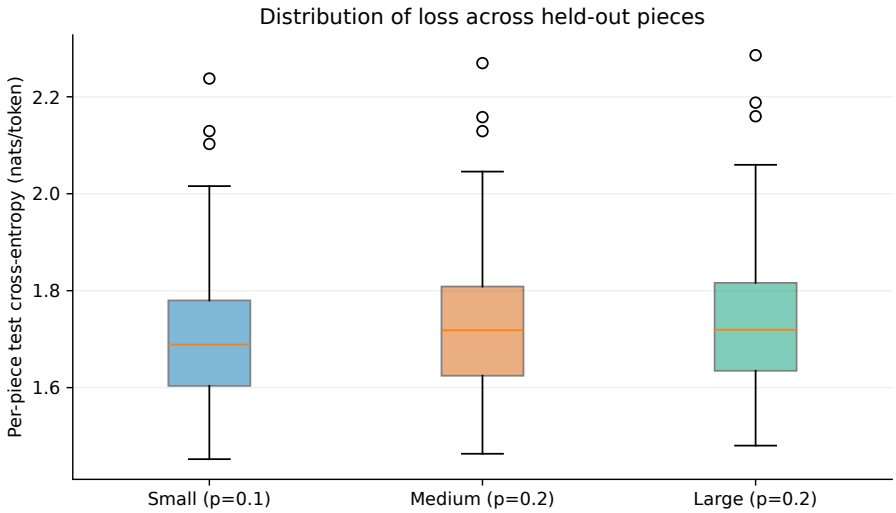


Figure 3: Test-loss distributions for the selected settings: small dropout 0.1, medium/large dropout 0.2. Each model is scored on the same 177 pieces. These distributions give each piece equal weight, while the overall loss gives each token equal weight. They do not measure variation across training seeds.

Table 8: Exploratory checks on the same saved test predictions. Equal-piece loss gives every piece the same weight. The last column counts pieces where small has lower loss. These are not repeated training runs.

Model	Token-weighted loss	Equal-piece loss	Small wins / 177
Small	1.6935	1.7090	–
Medium	1.7179	1.7335	177
Large	1.7277	1.7432	177

Giving each piece equal weight produced the same ranking, and small had lower loss on all 177 matched pieces (Table 8). A few long pieces therefore did not explain its lead. These exploratory checks used the same fitted checkpoints, so they do not measure variation across training seeds or establish reliability on independent datasets. The distributions in Figure 3 must not be interpreted as repeated training runs.

Table 9: Per-event-family test cross-entropy for the selected configurations. Counts are shared across models.

Family	Targets	Small ($p = 0.1$)	Medium ($p = 0.2$)	Large ($p = 0.2$)
NOTE_ON	741,410	1.7608	1.8615	1.8960
NOTE_OFF	741,410	1.1307	1.1544	1.1721
TIME_SHIFT	1,999,264	1.6817	1.6804	1.6805
VELOCITY	741,410	2.2357	2.2538	2.2692
PEDAL	110,421	1.5642	1.5862	1.5194
EOS	177	20.5116	21.6459	20.2792

Table 9 shows differences hidden by the aggregate. After weighting each category's mean-loss difference by its share of targets, note starts accounted for about 0.0172 nats/token of medium's total gap of 0.0244 and 0.0231 nats/token of large's total gap of 0.0342 (71% and 68%, respectively). Their slight gains on time shifts, and large's gain on pedal events, did not offset note-related losses. This supports investigating pitch-pattern prediction, but does not establish an internal cause.

EOS accounted for only 177 targets (0.0041%). Its high mean loss contributed just 0.00084 nats/token to small's aggregate, so good average prediction coexisted with poor prediction of endings. Fixed training windows may have contributed: EOS can appear only as the last target of the final valid window, unlike interior tokens. This explanation remains untested. Termination checks and listening evaluation would be needed before claiming musical quality; neither was performed here.

10 Conclusions and next steps

Main conclusion

Larger models did not perform best within this two-hour A10 budget. Stronger dropout improved medium and large, but mainly limited late increases in validation loss; it did not reverse the ranking. Small's reduced training–validation gap did not produce better validation loss, showing why regularization must be judged by prediction on unseen data rather than gap reduction alone. These are useful negative results for this pipeline, not a general rule that smaller models are better.

What remains uncertain

One seed per setting, two dropout values, a shared learning rate, and reused validation windows limit the comparison. Equal training time produced different update counts and exposure, and ten-minute checks may have missed the best checkpoint. Small, medium, and large processed about 125.4, 40.1, and 15.2 corpus equivalents, respectively. These are ratios from sampling with replacement, not new data or ordered epochs. Data-constrained language-model research finds diminishing returns from repetition [12], motivating an experiment on data diversity. Its numerical thresholds do not transfer directly to these MIDI windows. Neither the curves nor the 177-piece result separates the effects of capacity, optimization, and training variation.

First follow-up: training-only pitch augmentation

A useful next experiment would use small as the baseline and transpose note starts and matching endings together, rejecting shifts outside the playable range. This would vary absolute pitch while preserving intervals. The approach assumes that the resulting keys and registers would remain useful training examples. Music Transformer uses pitch/time augmentation in its piano experiments [10, Section 4.2]. This provides a relevant precedent, not evidence that the proposed change would improve loss.

Preferred architecture experiment: small with relative attention

Music Transformer reports benefits on expressive piano event sequences [10], supporting a test of relative event positions instead of only absolute positions. Hoffmann et al. discuss MoE and retrieval as alternatives [9, Section 2], but do not establish their suitability here. MoE adds conditional capacity and routing complexity [13]. The larger models already fitted the training data well, making extra capacity a less direct response to the observed generalization problem. Relative attention is a testable choice, not a proven remedy. Event distance is not musical time, and extra attention work may reduce throughput.

Comparison plan

Augmentation alone and relative attention alone should be compared against repeated small baselines, keeping the context length of 1,024, dropout, two-hour budgets, and tuning effort comparable across several seeds. The main criterion should be validation-selected loss, with tokens, speed, memory, and parameters also reported. An equal-token comparison would help separate speed from learning efficiency. Secondary checks should examine note events, EOS, and continuations generated from matching prompts. Since the test results have been inspected, further selection should use validation, and independent confirmation would require a new holdout. All these experiments remain proposed.

References

[1] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1810.12247.

[2] Magenta. The MAESTRO dataset, version 3.0.0, 2026. URL https://magenta.tensorflow.org/datasets/maestro. Dataset documentation, accessed 8 November 2026.

[3] HuggingFaceTB. The smol training playbook: The secrets to building world-class LLMs, 2025. URL https://huggingface.co/spaces/HuggingFaceTB/smol-training-playbook. Practitioner guide; accessed 12 November 2026.

[4] NVIDIA. NVIDIA A10 tensor core GPU, 2026. URL https://www.nvidia.com/en-us/data-center/products/a10-gpu/. Product specifications, accessed 8 November 2026.

[5] Andrej Karpathy. nanoGPT, 2026. URL https://github.com/karpathy/nanoGPT. Source implementation, accessed 8 November 2026.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. URL https://arxiv.org/abs/1706.03762.

[7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[8] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org/contents/ml.html. Chapter 5.

[9] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2203.15556.

[10] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer: Generating music with long-term structure. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1809.04281.

[11] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL https://jmlr.org/papers/v15/srivastava14a.html.

[12] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/9d89448b63ce1e2e8dc7af72c984c196-Abstract-Conference.html.

[13] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.